

ANNEXE TECHNIQUE

BouzuSec Scanner — SAE 401

Documentation technique approfondie — Scoring, Algorithmes, Rapports PDF

BouzuCorp — Johan Abdallah · Mohamed Droussi · Alexandre Samperez · Walid Badaoui · Nicolas
Nonnenmacher

1. Système de Scoring ANSSI — Détail Complet

La présentation expose la métaphore de la batterie et la formule hybride. Cette section détaille la mécanique interne, paramètre par paramètre, telle qu'elle est implémentée dans `scoring.py`.

1.1 Les 5 axes du radar et leur poids

Chaque axe est calculé indépendamment par une fonction dédiée, puis agrégé. Le tableau ci-dessous résume les plages de score, les outils qui les alimentent et les types de failles détectées.

Axe du radar	Fonction Python	Outil(s)	Ce qui est mesuré
Données Publiques (OSINT)	<code>evaluer_osint()</code>	Whois + WhatWeb	Versions exposées, e-mails, technologies identifiées
Portes réseau	<code>evaluer_reseau()</code>	Nmap	Nombre de ports ouverts, ports critiques (21, 445, 3306)
Dossiers cachés	<code>evaluer_enum()</code>	GoBuster	Fichiers <code>.env</code> , <code>.git</code> , backups, panneaux admin exposés
Configuration serveur	<code>evaluer_config()</code>	Nikto	Headers HTTP manquants, directory listing, logiciels obsolètes
Code applicatif	<code>evaluer_app()</code>	Nikto	Injections SQL/XSS, CSRF, exécution de code à distance

1.2 La formule hybride du maillon faible

La formule présentée en diaporama est :

```
risque_global = (pire_risque * 0.5) + (moyenne_5_axes * 0.5)
energie_sante = 100 - (risque_global * 9) ← plafonné entre 5 % et 100 %
note_sur_10 = round(energie_sante / 10, 1)
```

Le coefficient 0,5 / 0,5 encode le principe ANSSI : un seul maillon faible dégrade radicalement le score global, mais une infrastructure globalement propre avec une seule faille reste pénalisée proportionnellement. Le multiplicateur 9 (et non 10) évite d'atteindre 0 % pour un score de 10/10 sur un seul axe — il laisse toujours un peu de vert, conformément aux recommandations de communication non-alarmiste de l'ANSSI.

1.3 La matrice de niveaux ANSSI

En fonction de `risque_global`, le code attribue une couleur RGB et un libellé :

risque_global	Niveau ANSSI	Barres batterie	Couleur RGB
≥ 7.5	RISQUE CRITIQUE	1 barre	(229, 57, 53) — Rouge
$5 \leq x < 7.5$	RISQUE ÉLEVÉ	2 barres	(230, 126, 34) — Orange
$3 \leq x < 5$	RISQUE MODÉRÉ	3 barres	(253, 216, 53) — Jaune
$1 \leq x < 3$	RISQUE FAIBLE	4 barres	(39, 174, 96) — Vert
< 1	RISQUE TRÈS FAIBLE	4 barres	(39, 174, 96) — Vert

2. Détail des Algorithmes de Scoring

2.1 evaluer_osint() — Axe Données Publiques

Analyse le texte brut retourné par Whois et WhatWeb. Le scoring est additif : chaque condition indépendante s'additionne, les bonnes pratiques retirent des points (jusqu'à 0 minimum).

```
# scoring.py - evaluer_osint(texte)
def evaluer_osint(texte):
    t = texte.lower()
    risque = 0

    # E-mail exposé dans le Whois → risque phishing
    if 'risque détecté' in t and ('e-mail' in t or 'mail' in t):
        risque += 2
    if 'phishing' in t:
        risque += 1

    # Whois anonymisé = bonne pratique → bonus
    if 'anonymisation' in t or 'bonne pratique' in t:
        risque = max(0, risque - 1)

    # Version de serveur exposée (ex : Apache/2.4.65) → CVE exploitable
    if 'version' in t and ('apache' in t or 'php' in t or 'nginx' in t):
        risque += 4 # +4 pts car permet ciblage précis
    elif 'apache' in t or 'php' in t or 'nginx' in t:
        risque += 2 # +2 pts sans version (surface réduite)

    # CMS détecté = surface élargie (plugins vulnérables)
    if 'wordpress' in t: risque += 3
    if 'phpmyadmin' in t: risque += 5 # Panneau BDD exposé = critique
    if 'debian' in t: risque += 1 # Info système mineur

    return min(risque, 10) # Plafond à 10
```

2.2 evaluer_reseau() — Axe Portes Réseau

Analyse la sortie textuelle de Nmap. Deux vecteurs : le volume de ports ouverts et la présence de ports à haut risque.

```
# scoring.py - evaluer_reseau(texte)
def evaluer_reseau(texte):
    t = texte.lower()
    risque = 0

    # Aucun port = infrastructure hermétique → score 0
    if 'hermétique' in t or 'aucun point d\'entrée' in t:
        return 0

    # Regex : extrait le nombre de portes depuis le texte vulgarisé
    match_nbportes = re.search(r'(\d+)\s*portes?\s*de\s*communication', t)
    if match_nbportes:
        nb = int(match_nbportes.group(1))
        if nb > 10: risque += 5
        elif nb > 5: risque += 3
        elif nb > 2: risque += 2
        else: risque += 1

    # Ports critiques (pondération par gravité métier)
    if 'ftp' in t or 'transfert de documents' in t: risque += 4
    if 'smb' in t or '445' in t: risque += 5 # Ransomware
    if 'mysql' in t or '3306' in t: risque += 5 # BDD exposée
    if 'ssh' in t or 'maintenance technique' in t: risque += 1

    return min(risque, 10)
```

2.3 evaluer_enum() — Axe Dossiers Cachés

Analyse la sortie de GoBuster. Les failles sont pondérées selon leur gravité CVSS estimée. C'est l'axe qui peut générer les scores les plus élevés (exposition de .env ou .git = +5 pts immédiatement).

```
# scoring.py - evaluer_enum(texte) [extrait]

# Fuites CRITIQUES (CVSS 9+) - chacune ajoute 4 ou 5 pts
if 'saint graal' in t or 'fichiers d\'environnement' in t: risque += 5
if 'code source' in t and 'git' in t: risque += 5
if 'aspirations de bases' in t or 'dump' in t: risque += 5
if 'clés de chiffrement' in t: risque += 5
if 'mots de passe hachés' in t: risque += 4

# Panneaux admin (CVSS 6-7)
if 'portes d\'administration' in t: risque += 3
if 'interface de gestion' in t: risque += 4

# Codes HTTP dangereux - chaque code 200 (accès direct) = +2 pts
nb_200 = t.count('code 200') # accès direct autorisé
nb_500 = t.count('code 500') # erreur = mauvaise config
risque += nb_200 * 2
risque += nb_500 * 1

# Bonus si beaucoup d'éléments critiques
nb_elements = t.count('concernant l\'élément critique')
if nb_elements >= 5: risque += 2
elif nb_elements >= 3: risque += 1

return min(risque, 10)
```

2.4 evaluer_config() et evaluer_app() — Axes Nikto

Les deux fonctions analysent la même sortie Nikto mais sous deux angles distincts. Un tag spécial [STATS_NIKTO] injecté par vuln_scan.py communique le nombre brut de découvertes.

```
# vuln_scan.py injecte ce tag invisible pour le scoring :
texte_final += f'\n\n[STATS_NIKTO] Découvertes brutes : {nb_decouvertes_brutes} |
Catégories : {nb_categories}'

# scoring.py - evaluer_config() extrait ces stats :
match_brut = re.search(r'\[STATS_NIKTO\] Découvertes brutes : (\d+)', texte)
if match_brut:
    nb_brut = int(match_brut.group(1))
    if nb_brut > 20: risque += 3
    elif nb_brut > 10: risque += 2
    elif nb_brut > 5: risque += 1

# evaluer_app() se concentre sur les failles OWASP critiques :
if 'injection' in t and 'sql' in t: risque += 5
if 'xss' in t or 'cross-site scripting' in t: risque += 4
if 'csrf' in t: risque += 4
if 'remote code' in t or 'exécution' in t: risque += 5
```

Point clé : la séparation evaluer_config / evaluer_app permet d'avoir deux axes distincts sur le radar même si les deux s'appuient sur Nikto. Config mesure les en-têtes et mauvaises configurations (réponse du serveur), tandis qu'App mesure les vulnérabilités dans le code applicatif lui-même.

3. Algorithmes des Modules de Scan

3.1 enumeration.py — Logique GoBuster et croisement métier

GoBuster retourne des lignes brutes contenant des codes ANSI qui doivent être nettoyés. La logique de croisement est au coeur de la valeur ajoutée de BouzuSec : elle traduit des chemins techniques en alertes métier.

```
# enumeration.py — Pipeline de traitement GoBuster

# 1. Nettoyage des codes couleur ANSI de la sortie terminale
sortie_propre = re.sub(r'\x1b\[[0-9;]*[a-zA-Z]', '', res.stdout)

# 2. Extraction chemin + code HTTP par regex
for ligne in lignes:
    if 'Status:' in ligne:
        dossier = ligne.split()[0].strip().lower().replace('/', '')
        dossier = re.sub(r'^\w.\-', '', dossier) # sanitation
        match_code = re.search(r'Status:\s*(\d{3})', ligne)
        code_http = match_code.group(1) if match_code else 'Inconnu'
        chemins_trouves[dossier] = code_http

# 3. Croisement avec ANALYSE_METIER (16 catégories de failles)
for chemin, code_http in chemins_trouves.items():
    for categorie, data in ANALYSE_METIER.items():
        for mot_cle in data['mots_cles']:
            if mot_cle in chemin: # correspondance partielle
                etat_francais = traduire_code_http(code_http)
                texte_alerte = f'{data["titre"]} ({chemin}) : {data["explication"]} | {etat_francais}'
                if texte_alerte not in alertes_generees: # déduplication
                    alertes_generees.append(texte_alerte)
```

Dictionnaire ANALYSE_METIER — 16 catégories

Chaque catégorie associe des mots-clés aux chemins GoBuster, un titre vulgarisé et une explication métier. Exemple :

```
ANALYSE_METIER = {
    'fichier_env': {
        'mots_cles': ['.env', 'config.env', 'docker-compose.env'],
        'titre': 'Fichiers d\'environnement et secrets',
        'explication': 'Le Saint Graal des pirates. Ce fichier contient les mots de passe de votre base de données et vos clés d\'API...'
    },
    'code_git': { 'mots_cles': ['.git', '.svn'], ... },
    'sauvegardes_bdd': { 'mots_cles': ['backup', 'dump.sql', 'db.sql'], ... },
    # ... 13 autres catégories : cles_ssh, gestionnaire_bdd, conteneurs_docker,
    # journaux_logs, documentation_api, dossiers_upload, etc.
}
```

3.2 scanner.py — Adaptation du scan Nmap selon le mode

```
# scanner.py — faire_nmap(domaine, mode_scan)

if mode_scan == 'simple':
    commande = ['nmap', '-T4', '--top-ports', '100', domaine]
    # Rapide : sonde uniquement les 100 ports les plus fréquents
else:
    commande = ['nmap', '-T4', '-p-', domaine]
    # Complet : sonde l'intégralité des 65 535 ports

# Traduction des ports en langage métier :
ANALYSE_PORTS = {
    '21': 'Porte FTP — vol de documents internes',
    '22': 'Porte SSH — cible de bruteforce',
    '445': 'Porte SMB — risque ransomware critique',
    '3306': 'Porte MySQL — base de données exposée',
}
```

3.3 vuln_scan.py — Tuning adaptatif de Nikto

L'orchestrateur analyse les résultats Phase 1 pour choisir le profil de scan Nikto le plus pertinent :

```
# orchestrateur.py - sélection dynamique du mode Nikto

args_nikto = 'avance' # profil par défaut

if 'cloudflare' in technos or 'waf' in technos:
    args_nikto = 'furtif' # Nikto en mode lent avec pauses
    lancer_gobuster = False # GoBuster désactivé (serait bloqué)

elif 'phpmyadmin' in technos and '3306' in ports:
    args_nikto = 'avance_phpmyadmin' # Tuning 9 : focus sur BDD

elif 'wordpress' in technos and '3306' in ports:
    args_nikto = 'avance_wp_sql' # Focus WordPress + injections SQL

# Dans vuln_scan.py, chaque mode déclenche une commande différente :
if mode_scan == 'simple':
    commande = ['nikto', '-h', domaine, '-Tuning', '123', '-maxtime', '45s']
elif mode_scan == 'furtif':
    commande = ['nikto', '-h', domaine, '-Pause', '2', '-maxtime', '3m']
else: # avance
    commande = ['nikto', '-h', domaine, '-maxtime', '3m']
```

4. Orchestrateur — Flux d'Exécution et Parallélisme

L'orchestrateur (orchestrateur.py) est le chef d'orchestre qui pilote les 5 phases en utilisant `concurrent.futures.ThreadPoolExecutor` pour maximiser la performance.

4.1 Flux en 5 phases

Phase	Statut JSON (%)	Outils lancés	Action
1	5 → 10 %	Whois + WhatWeb + Nmap	3 threads parallèles — reconnaissance
2	40 → 50 %	Nikto + GoBuster	Décision adaptative selon résultats Phase 1
3	80 → 90 %	scoring.py	Calcul du score global ANSSI sur les 5 axes
4	90 %	generateur_pdf.py	Génération rapport_simple.pdf
5	90 → 100 %	generateur_pdf.py	Génération rapport_avance.pdf

4.2 Condition de déclenchement du scan web

```
# orchestrateur.py — détection de la présence d'un serveur web
technos = str(resultat_whatweb).lower()

presence_web = (
    'http' in technos or 'apache' in technos or 'nginx' in technos
    or 'cloudflare' in technos or len(technos) > 15
)

# Si aucun serveur web → Nikto et GoBuster ne sont pas lancés
# Exemple : serveur SSH pur → seuls Nmap et Whois sont pertinents
if not presence_web and '22' in ports:
    alertes_tactiques += 'Alerte : serveur pur SSH. Tests web annulés.\n\n'
```

4.3 Communication en temps réel avec le front-end

L'orchestrateur Python écrit un fichier JSON à chaque étape. Le JavaScript (index.php) le lit toutes les secondes par polling pour mettre à jour la barre de progression.

```
# orchestrateur.py — maj_statut()
def maj_statut(pourcentage, message):
    with open('statut_audit.json', 'w', encoding='utf-8') as f:
        json.dump({'pourcentage': pourcentage, 'message': message}, f)

# index.php — polling côté client (JavaScript)
const pollInterval = setInterval(() => {
    fetch('statut_audit.json?t=' + new Date().getTime())
        .then(r => r.json())
        .then(data => {
            fillElement.style.width = data.pourcentage + '%';
            messageElement.innerText = data.message;
            if (data.pourcentage >= 100) {
                clearInterval(pollInterval);
                window.location.href = 'index.php?scan=termine&cible=' + cible;
            }
        });
}, 1000); // polling toutes les secondes
```

5. Génération des Rapports PDF — Architecture Interne

La présentation montre les pages finales. Cette section détaille la mécanique de `generateur_pdf.py`, les choix de rendu et le pipeline de création.

5.1 Architecture de la bibliothèque et encodage

```
# generateur_pdf.py — Bibliothèques utilisées
from fpdf import FPDF          # Génération PDF pure Python
import matplotlib              # Tracé du radar (graphe polaire)
import numpy as np             # Calculs angulaires pour le radar
matplotlib.use('Agg')          # Rendu sans interface graphique (serveur)

# Fonction de nettoyage encodage latin-1 (limitation de FPDF v1)
def _sec(texte):
    for src, dst in [('\u2019', '"'), ('\u2013', '-'), ('\u2022', '-')]:
        texte = texte.replace(src, dst)
    return texte.encode('latin-1', 'replace').decode('latin-1')
```

5.2 Génération du graphe radar

Le radar est tracé en matplotlib, exporté en PNG transparent, inclus dans le PDF, puis le fichier temporaire est supprimé.

```
# generateur_pdf.py — _radar(scores, nom_fichier)
def _radar(scores, nom_fichier='radar_tmp.png'):
    labels = np.array(['Données Publiques', 'Portes (Réseau)',
                       'Dossiers Cachés', 'Configuration', 'Code Applicatif'])
    angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False)

    # Fermeture du polygone (dernier point = premier)
    s = np.concatenate((scores, [scores[0]]))
    a = np.concatenate((angles, [angles[0]]))

    fig, ax = plt.subplots(figsize=(5.5, 5.5), subplot_kw=dict(polar=True))
    ax.fill(a, s, color='#e67e22', alpha=0.4) # remplissage orange semi-transparent
    ax.plot(a, s, color='#d35400', linewidth=2) # contour orange foncé
    ax.set_ylim(0, 10)                        # échelle 0 à 10
    plt.savefig(nom_fichier, transparent=True, bbox_inches='tight')
    plt.close()

# Intégration dans le PDF :
pdf.image('radar_tmp.png', x=45, y=150, w=120) # centré, 120mm de large
os.remove('radar_tmp.png')                    # nettoyage immédiat
```

5.3 Rendu de la batterie (indicateur visuel)

L'icône batterie est dessinée entièrement avec les primitives de FPDF (rectangles) sans image externe.

```
# generateur_pdf.py — _batterie(pdf, x, y, barres_actives, couleur)
def _batterie(pdf, x, y, barres_actives, couleur):
    # Coque extérieure (fond sombre)
    pdf.set_fill_color(40, 40, 40)
    pdf.rect(x, y, 60, 25, 'F') # 60mm × 25mm
    # Borne positive (petit rectangle à droite)
    pdf.rect(x + 60, y + 7, 4, 11, 'F')
    # Fond blanc intérieur
    pdf.set_fill_color(255, 255, 255)
    pdf.rect(x + 2, y + 2, 56, 21, 'F')
    # 4 barres : colorées si actives, grises sinon
    for i in range(4):
        if i < barres_actives:
            pdf.set_fill_color(*couleur) # couleur ANSSI
        else:
            pdf.set_fill_color(230, 230, 230)
        pdf.rect(x + 3 + i * 14, y + 3, 12.5, 19, 'F')
```


5.4 Pipeline de nettoyage avant PDF

Les tags [STATS_NIKTO] injectés pour le scoring doivent être retirés avant la génération PDF :

```
# orchestrateur.py - nettoyage avant rendu PDF
import re
diagnostic_complet_pdf = re.sub(
    r'\n*\[STATS_NIKTO\].*$',  # supprime tout depuis le tag jusqu'à la fin
    '',
    diagnostic_complet,
    flags=re.DOTALL
)
# Le texte envoyé au PDF est propre, sans méta-données de scoring
```

5.5 Structure des deux rapports PDF

Page	Rapport Simple (2-3 p.)	Rapport Avancé (6-7 p.)
1	Dashboard : batterie + radar + note ANSSI	Identique (page commune _page_couverture)
2	Vue par axe (FAIBLE/MODÉRÉ/ÉLEVÉ/CRITIQUE) + 3 recommandations prioritaires	Résultats Whois + WhatWeb (Reconnaissance & Identité)
3	—	Résultats Nmap + GoBuster commentés
4	—	Plan d'action (URGENCE / PRIORITÉ / AMÉLIORATION)
5	—	Méthodologie ANSSI + limites du diagnostic
6	—	Lexique cybersécurité vulgarisé pour la direction

6. Installation et Utilisation de l'Environnement

6.1 Prérequis

- Docker Desktop installé (docker.com) — inclut Docker Compose
- 4 Go de RAM minimum sur la machine hôte
- Connexion Internet pour le premier build (téléchargement des images)

6.2 Démarrage en 3 commandes

```
# Cloner le dépôt GitHub
git clone https://github.com/spalexandre13/BouzuSec.git
cd BouzuSec

# Construire et démarrer les 2 conteneurs
docker compose up -d --build
# Premier build : 5-10 minutes (Nmap, Nikto, GoBuster, WhatWeb, Python libs)

# Arrêt
docker compose down
```

6.3 Accès aux interfaces

Service	URL	Description
BouzuSec Scanner	http://localhost:9090	Interface de l'outil d'audit
Site vulnérable de test	http://localhost:8888	Cible avec failles intentionnelles

6.4 Lancer un scan de test

- Ouvrir http://localhost:9090 dans le navigateur
- Entrer site-test dans le champ URL (nom DNS interne Docker)
- Sélectionner Mode Avancé pour une analyse complète
- Cliquer sur Diagnostiquer — durée estimée : 2 à 5 minutes
- Télécharger les deux rapports PDF depuis la page de résultats

6.5 Architecture réseau Docker

```
# docker-compose.yml — réseau bridge isolé
services:
  bouzusec:
    # Scanner BouzuSec
    build: { context: ./scanner }
    ports: ['9090:80'] # Exposé sur le port 9090 de l'hôte
    networks: [bouzusec-net]
    depends_on: [site-test]

  site-test:
    # Site vulnérable
    build: { context: ./site-test }
    ports: ['8888:80']
    networks: [bouzusec-net]

networks:
  bouzusec-net:
    # Bridge interne — scanner → site-test via DNS
    driver: bridge
```

6.6 Dépannage rapide

Problème	Commande de diagnostic
Le scan ne démarre pas	docker compose logs bouzusec

Problème	Commande de diagnostic
Site de test inaccessible	<code>docker compose logs site-test</code>
Port déjà utilisé	Modifier le port dans <code>docker-compose.yml</code>
Rebuild après modif de code	<code>docker compose down && docker compose up -d --build</code>

BouzuCorp — SAE 401 — Document confidentiel